LifeSamadhan Interview Questions and Answers

Generated on: 2026-04-29

This document contains the interview-style questions and answers discussed for the LifeSamadhan project. It covers project purpose, technical architecture, JWT, Razorpay, OTP, SMTP, scalability, and performance handling.

Note: Real credentials such as SMTP passwords, Razorpay secrets, JWT secrets, and database passwords should not be shared in interviews or committed to source control. Use placeholders or environment variables.

---

1. Explain The Full Project In Brief

Question

Explain the full project with frontend and both backends, including all features in brief.

Answer

LifeSamadhan is a full-stack local service booking platform. It connects customers with verified local service providers and gives admins control over provider verification, service categories, services, locations, and approvals.

The project has one React frontend and two backend implementations:

- React frontend
- Spring Boot backend
- ASP.NET Core .NET backend

Frontend

The frontend is built with React, Vite, Axios, and React Router.

Main frontend features:

- Public pages: Home, About Us, Contact Us, Login, Register.
- JWT-based authentication.
- Role-based dashboards for Admin, Customer, and Service Provider.
- Admin can manage provider types, categories, services, locations, providers, and skill approvals.
- Customer can search providers, book services, cancel bookings, pay online, rate providers, and update profile.
- Provider can view assigned jobs, accept or reject jobs, start work with OTP, complete jobs, and view earnings.
- Real-time notifications are supported using SignalR for .NET and STOMP/SockJS WebSocket for Spring Boot.

Spring Boot Backend

The Spring Boot backend is built using Java 17, Spring Boot, Spring Data JPA, Spring Security, JWT, MySQL, WebSocket, SMTP, and Razorpay.

Main backend features:

- User registration and login.
- JWT authentication.
- Role-based authorization.
- Controller, service, repository architecture.
- MySQL database with JPA entities.
- Admin, customer, provider, payment, rating, service, category, location, and public APIs.
- OTP email using SMTP.
- Razorpay payment order creation and verification flow.
- WebSocket real-time updates.
- Scheduled cleanup for stale assignments.

.NET Backend

The .NET backend is built using ASP.NET Core .NET 8, Entity Framework Core, MySQL, JWT, SignalR, Razorpay, SMTP email, and Swagger.

Main backend features:

- JWT authentication and role authorization.
- Entity Framework Core database access.
- SignalR notifications.
- Service booking and assignment workflow.
- OTP generation and email sending.
- Razorpay payment integration.
- Admin provider verification and skill approval.
- Background cleanup for timed-out assignments.

---

2. What Problem Is LifeSamadhan Solving?

Question

Tell me from an interviewer point of view: what problem is it solving?

Answer

LifeSamadhan solves the problem of finding trusted local service providers quickly and safely.

In daily life, customers need services like repair, maintenance, cleaning, plumbing, electrical work, and other local services. Usually, they face problems such as:

- Not knowing which provider is available nearby.
- Difficulty checking whether a provider is genuine.
- No proper booking and tracking system.
- No transparent pricing.
- No secure verification before service starts.
- No easy rating and feedback system.
- Providers also struggle to get genuine service requests in an organized way.

LifeSamadhan provides a digital platform where customers can search verified providers, book services, track status, pay online, and rate service quality. Providers can manage jobs and earnings, while admins can verify providers and control the service ecosystem.

Short Interview Answer

LifeSamadhan is a service provider management and booking platform. It solves the problem of unreliable and unorganized local service booking by connecting customers with verified local providers based on category and location. Customers can book services, track work, pay online, and give feedback, while providers and admins manage the service workflow efficiently.

---

3. How Is LifeSamadhan Different From Urban Company?

Question

There are already many platforms like Urban Company. How is this different?

Answer

LifeSamadhan is not trying to replace Urban Company directly. Urban Company is a large public marketplace, while LifeSamadhan is designed as a customizable local service management platform.

Key differences:

- LifeSamadhan can be deployed for a specific city, district, society, campus, local business, or institution.
- It gives local admins direct control over providers, services, locations, and approvals.
- Provider skills can be approved or rejected by admin.
- Providers must be verified before they are trusted in the system.
- OTP is required before the provider starts service.
- Hourly-rate-based pricing is supported.
- The same application has both Spring Boot and .NET backend implementations, showing technology flexibility.

Short Interview Answer

Urban Company is a large public marketplace. LifeSamadhan is a customizable local service management platform where an admin can control providers, services, locations, verification, bookings, payments, and ratings for a specific local ecosystem.

---

4. Explain The Project Technically In Programs

Question

Help me explain it technically in programs.

Answer

LifeSamadhan follows a full-stack architecture:

```text
React Frontend
   |
Axios API calls
   |
Spring Boot Controllers
   |
```

```
Service Layer
    |
Repository Layer
    |
MySQL Database
```

Frontend Flow

Main frontend files:

- `App.jsx`: Handles routes.
- `Login.jsx`: Handles login UI and login request.
- `Register.jsx`: Handles customer/provider registration.
- `AuthContext.jsx`: Stores logged-in user and role.
- `utils/api.js`: Axios instance that attaches JWT token.
- Admin, customer, and provider dashboards handle role-specific workflows.

Login flow:

```text
Login.jsx
    -> calls login API
    -> POST /api/auth/login
    -> receives token and role
    -> stores token in localStorage
    -> redirects user based on role
```

Backend Flow

Spring Boot uses:

```text
Controller -> Service -> Repository -> Database
```

Example:

```text
AuthController
    -> receives login/register request
UserService / PasswordService / JwtUtils
    -> handles business logic
UserRepository
    -> communicates with MySQL
User entity
    -> maps to users table
```

Role-Based Flow

Roles:

```text
ADMIN
CUSTOMER
```

SERVICEPROVIDER
‘‘‘

Admin controls master data and verification. Customer books services. Provider accepts, starts, and completes jobs.

Short Interview Answer

Technically, LifeSamadhan is a role-based full-stack service booking system. React handles the UI and sends Axios API calls. Spring Boot handles REST APIs using controller-service-repository architecture. MySQL stores users, providers, services, bookings, payments, and ratings. JWT handles authentication, WebSocket handles real-time updates, SMTP sends OTP emails, and Razorpay handles payment.

---

5. Explain JWT In The Project

Question

Explain JWT in the project. How is it used and what is the real working of it?

Answer

JWT is used as a signed login token. After successful login, the backend gives a token to the frontend. The frontend sends this token with every protected API request.

JWT Login Flow

‘‘‘text
User enters email and password
   |
Frontend calls POST /api/auth/login
   |
AuthController validates credentials
   |
JwtUtils generates token
   |
Frontend stores token in localStorage
   |
Every protected request sends Authorization: Bearer <token>
‘‘‘

JWT Content

In the project, token is generated using:

‘‘‘java
jwtUtils.generateToken(user.getEmail(), user.getId(), user.getRole());
‘‘‘

The token contains:

‘‘‘json
{
  "sub": "user@email.com",
  "user_id": 5,

```
  "role": "CUSTOMER",
  "iat": "issued time",
  "exp": "expiry time"
}
```

Frontend Usage

In `utils/api.js`, Axios adds the token:

```js
config.headers.Authorization = `Bearer ${token}`;
```

Backend Validation

`CustomJwtFilter`:

- Reads the `Authorization` header.
- Extracts the token after `Bearer`.
- Extracts email from token.
- Loads user from database.
- Validates token signature and expiry.
- Sets authentication in Spring Security context.

Role Authorization

`SecurityConfiguration` protects APIs:

```java
.requestMatchers("/api/admin/**").hasRole("ADMIN")
.requestMatchers("/api/customer/**").hasRole("CUSTOMER")
.requestMatchers("/api/provider/**").hasRole("SERVICEPROVIDER")
```

Short Interview Answer

JWT is used for stateless authentication. After login, the backend generates a signed token containing email, user id, role, issue time, and expiry. The frontend stores it and sends it in the Authorization header. The backend validates it in a custom JWT filter and Spring Security uses the role to protect admin, customer, and provider APIs.

---

6. Explain Razorpay API In The Project

Question

Explain Razorpay API used in the project. How is it used and where?

Answer

Razorpay is used for online payment after service completion.

Main files:

- Frontend: `src/api/payment.js`

- Frontend: ‘CustomerDashboard.jsx‘
- Backend: ‘PaymentController.java‘
- Backend: ‘PaymentService.java‘
- Config: ‘application.properties‘

Payment Flow

“‘text
Customer clicks Pay
  |
Frontend calls POST /api/payment/create-order/{assignmentId}
  |
Backend creates Razorpay order
  |
Backend saves payment as PENDING
  |
Frontend opens Razorpay checkout popup
  |
Customer completes payment
  |
Razorpay returns order id, payment id, signature
  |
Frontend calls POST /api/payment/verify
  |
Backend marks payment completed and request paid
“‘

Backend Order Creation

‘PaymentService‘ creates Razorpay client:

“‘java
RazorpayClient razorpay = new RazorpayClient(razorpayKeyId, razorpayKeySecret);
“‘

It creates an order:

“‘java
orderRequest.put("amount", amount.multiply(new BigDecimal(100)).intValue());
orderRequest.put("currency", "INR");
Order order = razorpay.orders.create(orderRequest);
“‘

Razorpay expects amount in paise, so INR amount is multiplied by 100.

Frontend Checkout

Frontend receives:

“‘text
orderId
amount
keyId
“‘

Then it opens:

```js
const rzp = new window.Razorpay(options);
rzp.open();
```

Important Note

The Spring Boot version currently marks payment completed after receiving payment details. For production, Razorpay signature should be verified using HMAC with the Razorpay secret. The .NET backend already contains signature verification logic.

Short Interview Answer

Razorpay is integrated for online service payment. The backend creates a Razorpay order and stores a pending payment record. The frontend opens Razorpay checkout. After payment, Razorpay returns payment details, which frontend sends to the backend. The backend updates the payment status and marks the service request as paid.

---

7. Explain OTP In The Project

Question

Explain OTP in your project. How is it generated and how does it verify whether it is correct or not?

Answer

OTP is used to verify that the service provider has actually reached the customer before starting the service.

Where OTP Is Stored

OTP is stored in ‘ServiceAssignment‘:

```java
@Column(name = "otp")
private String otp;
```

OTP Generation

When a customer books a service, backend creates a service assignment and generates OTP:

```java
.otp(String.format("%04d", new java.util.Random().nextInt(10000)))
```

This creates a 4-digit OTP:

```text
7    -> 0007
45   -> 0045
583  -> 0583
9821 -> 9821
```

OTP Delivery

The backend sends OTP to customer email using SMTP:

```java
emailService.sendOtpEmail(
    customerUser.getEmail(),
    savedAssignment.getOtp(),
    providerUser.getName(),
    serviceName
);
```

OTP Verification

Provider enters OTP in provider dashboard. Frontend calls:

```text
POST /api/provider/assignment/{assignmentId}/start
```

Backend sanitizes OTP:

```java
String otp = otpPayload.replaceAll("\\D", "");
```

Then ‘ProviderService‘ compares entered OTP with stored OTP:

```java
if (assignment.getOtp() == null || !assignment.getOtp().equals(otp)) {
    throw new RuntimeException("Invalid OTP");
}
```

If OTP matches:

```text
ACCEPTED -> STARTED
```

If OTP is wrong:

```text
Invalid OTP
```

Short Interview Answer

OTP is generated during service assignment, sent to the customer by email, stored in the database, and verified when the provider starts the job. The backend compares the provider-entered OTP with the stored OTP. If it matches, the service status becomes STARTED; otherwise it returns Invalid OTP.

---

8. Explain SMTP Used In The Project

Question

Explain SMTP used in the project.

Answer

SMTP stands for Simple Mail Transfer Protocol. It is used to send emails from the application to users.

In LifeSamadhan, SMTP is used to send service OTP emails to customers.

SMTP Configuration

In ‘application.properties‘:

‘‘‘properties
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=<sender-email>
spring.mail.password=<app-password>
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
‘‘‘

Meaning:

- ‘smtp.gmail.com‘: Gmail SMTP server.
- ‘587‘: SMTP TLS port.
- ‘username‘: sender email.
- ‘password‘: Gmail app password.
- ‘smtp.auth=true‘: authentication is required.
- ‘starttls.enable=true‘: secure mail transfer is enabled.

Code Usage

‘EmailService‘ uses Spring Boot ‘JavaMailSender‘:

‘‘‘java
private final JavaMailSender mailSender;
‘‘‘

It builds and sends the email:

‘‘‘java
SimpleMailMessage message = new SimpleMailMessage();
message.setFrom(fromEmail);
message.setTo(toEmail);
message.setSubject("Your Service OTP - LifeSamadhan");
message.setText("OTP: " + otp);
mailSender.send(message);
‘‘‘

Flow

‘‘‘text

```
Customer books service
   |
Backend creates assignment
   |
OTP is generated
   |
EmailService.sendOtpEmail() is called
   |
JavaMailSender connects to Gmail SMTP
   |
OTP email is sent to customer
```

Short Interview Answer

SMTP is used in LifeSamadhan to send OTP emails to customers. Spring Boot reads SMTP configuration from ‘application.properties‘, creates ‘JavaMailSender‘, and ‘EmailService‘ uses it to send the OTP through Gmail SMTP.

---

9. Is SMTP Built Into Spring Boot Like .NET?

Question

Does this mean SMTP is an inbuilt feature of Spring Boot for sending mail, like in .NET?

Answer

SMTP itself is not a Spring Boot feature. SMTP is a general email-sending protocol.

Spring Boot provides built-in support to use SMTP easily through:

```text
spring-boot-starter-mail
JavaMailSender
```

In the project, the dependency is:

```xml
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-mail</artifactId>
</dependency>
```

Spring Boot reads:

```properties
spring.mail.host
spring.mail.port
spring.mail.username
spring.mail.password
```

and configures ‘JavaMailSender‘.

In ASP.NET Core, similar work can be done with:

‘‘‘text
System.Net.Mail.SmtpClient
MailKit
‘‘‘

Short Interview Answer

SMTP is not specific to Spring Boot or .NET. It is a standard email protocol. Spring Boot provides built-in mail support using ‘spring-boot-starter-mail‘ and ‘JavaMailSender‘, just like .NET can send mail using ‘SmtpClient‘ or MailKit.

---

10. How To Handle 150 Requests At A Time?

Question

If asked that we have to handle 150 requests at a time, how can we handle it for this project?

Answer

For 150 concurrent requests, LifeSamadhan can handle it by using:

- Spring Boot Tomcat thread tuning.
- HikariCP database connection pooling.
- Optimized MySQL queries.
- JWT stateless authentication.
- Async email sending.
- Proper indexes.
- Load testing.

Tomcat Thread Tuning

Add in ‘application.properties‘:

‘‘‘properties
server.tomcat.threads.max=200
server.tomcat.threads.min-spare=20
server.tomcat.accept-count=100
‘‘‘

Database Connection Pool

Spring Boot uses HikariCP by default.

‘‘‘properties
spring.datasource.hikari.maximum-pool-size=30
spring.datasource.hikari.minimum-idle=10
spring.datasource.hikari.connection-timeout=30000
‘‘‘

150 requests do not require 150 database connections. Short requests reuse connections from the pool.

Why JWT Helps

JWT is stateless:

‘‘‘text
No server-side session
No login state stored in backend memory
Any backend instance can validate the token
‘‘‘

This makes horizontal scaling easier.

Short Interview Answer

For 150 concurrent requests, I would tune Tomcat threads, use HikariCP connection pooling, add database indexes, make email sending asynchronous, and test using JMeter or k6. Since our project uses JWT, it is stateless and can also be scaled horizontally behind a load balancer.

---

11. Maximum Requests Spring Boot Can Handle Alone

Question

Maximum how many requests can Spring Boot handle alone?

Answer

There is no fixed maximum. It depends on:

- CPU and RAM.
- API logic.
- Database speed.
- Number of DB connections.
- External calls such as SMTP and Razorpay.
- Tomcat thread pool.
- Query optimization.

By default, Spring Boot uses embedded Tomcat with a request thread pool. A common default maximum is around 200 worker threads, but this does not mean the app can complete unlimited requests quickly. If database pool has only 10 connections, DB-heavy APIs may wait.

Short Interview Answer

A single Spring Boot instance can usually handle hundreds of concurrent requests if APIs are optimized, but the exact number depends on hardware, database, and response time. In LifeSamadhan, the bottleneck may be MySQL, SMTP, or Razorpay, not Spring Boot itself.

---

12. Approximate Requests By Server Hardware

Question

Give approximate values of requests per server hardware size.

Answer

For a normal Spring Boot + MySQL CRUD project like LifeSamadhan, approximate values are:

Server Size	Approx Concurrent Requests	Approx Requests Per Second
1 vCPU, 1-2 GB RAM	20-50	20-80 RPS
2 vCPU, 4 GB RAM	80-150	80-200 RPS
4 vCPU, 8 GB RAM	200-500	200-600 RPS
8 vCPU, 16 GB RAM	500-1000+	600-1500 RPS
16 vCPU, 32 GB RAM	1000-3000+	1500-4000+ RPS

These are rough estimates. Exact values must be verified using load testing.

For LifeSamadhan

Because the project uses MySQL, JWT, SMTP, Razorpay, and WebSocket:

```text
2 vCPU / 4 GB RAM
   -> around 100 to 150 concurrent requests

4 vCPU / 8 GB RAM
   -> around 200 to 500 concurrent requests
```

If average response time is 500 ms:

```text
150 concurrent requests = about 300 requests per second
```

Short Interview Answer

A 2 vCPU and 4 GB RAM server can approximately handle 100 to 150 concurrent requests for this project if optimized. A 4 vCPU and 8 GB RAM server can approximately handle 200 to 500 concurrent requests. Exact numbers should be confirmed through JMeter or k6 load testing.

---

13. How To Handle More Than 1000 Requests Per Second?

Question

If I have 4 vCPU and 8 GB RAM and more than 1000 requests per second come, how can we handle it in our project? What do we have to add and where?

Answer

For more than 1000 requests per second, one 4 vCPU / 8 GB RAM Spring Boot server is usually not enough for this project. We should use horizontal scaling and supporting infrastructure.

Main Strategy

```text
Nginx / Load Balancer
```

```
      |
  ----------
 |    |    |
App 1  App 2  App 3
      |
    MySQL
      |
    Redis
```

Because authentication is JWT-based and stateless, any backend instance can validate the token.

1. Tune Tomcat

Add in ‘application.properties‘:

```properties
server.tomcat.threads.max=300
server.tomcat.threads.min-spare=50
server.tomcat.accept-count=500
```

2. Tune HikariCP

Add in ‘application.properties‘:

```properties
spring.datasource.hikari.maximum-pool-size=40
spring.datasource.hikari.minimum-idle=10
spring.datasource.hikari.connection-timeout=30000
```

Do not increase DB connections blindly. The database server must also be able to handle them.

3. Make Email Async

Email sending is slow because SMTP is an external service.

Add async support in ‘LifeSamadhanApiApplication.java‘:

```java
@SpringBootApplication
@EnableScheduling
@EnableAsync
public class LifeSamadhanApiApplication {
}
```

Then in ‘EmailService.java‘:

```java
@Async
public void sendOtpEmail(String toEmail, String otp, String providerName, String serviceName) {
    // email sending code
```

}
```

This prevents booking APIs from waiting for SMTP.

### 4. Add Cache

Cache data that changes rarely:

- Categories
- Services
- Locations
- Provider types

Add dependency:

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-cache</artifactId>
</dependency>
```

Enable caching:

```java
@EnableCaching
@SpringBootApplication
public class LifeSamadhanApiApplication {
}
```

Example:

```java
@Cacheable("locations")
public List<Location> getAllLocations() {
    return locationRepository.findAll();
}
```

For production, Redis is better than in-memory cache.

5. Add Database Indexes

Add indexes on frequently searched columns:

```text
users.email
users.role
service_assignments.provider_id
service_assignments.status
service_requests.customer_id
service_requests.status
provider_locations.location_id
provider_skills.service_id
service_providers.service_category_id
```
```

‘‘‘

This is important for login, dashboard, provider search, and assignment APIs.

6. Use Queue For Heavy Work

Move slow background tasks to queue:

- OTP email
- Notifications
- Payment post-processing

Possible tools:

‘‘‘text
RabbitMQ
Kafka
Redis Queue
‘‘‘

7. Load Testing

Use:

‘‘‘text
JMeter
k6
Postman Runner
‘‘‘

Measure:

- Response time.
- Error rate.
- CPU usage.
- Memory usage.
- DB connection usage.
- Slow queries.

Short Interview Answer

For more than 1000 requests per second, I would not depend on one Spring Boot server. I would run multiple Spring Boot instances behind a load balancer, move MySQL to a separate optimized server, add Redis caching, make SMTP and notifications async or queue-based, tune Tomcat and HikariCP, add database indexes, and verify capacity using JMeter or k6.

Quick Final Summary

LifeSamadhan is a role-based local service booking system. The frontend is built with React. The backend is implemented in Spring Boot and also in .NET. JWT secures APIs, Razorpay handles payments, OTP verifies service start, SMTP sends OTP emails, and WebSocket/SignalR handles real-time notifications. For scalability, the project can use thread tuning, DB pooling, indexes, caching, async processing, queues, and horizontal scaling behind a load balancer.